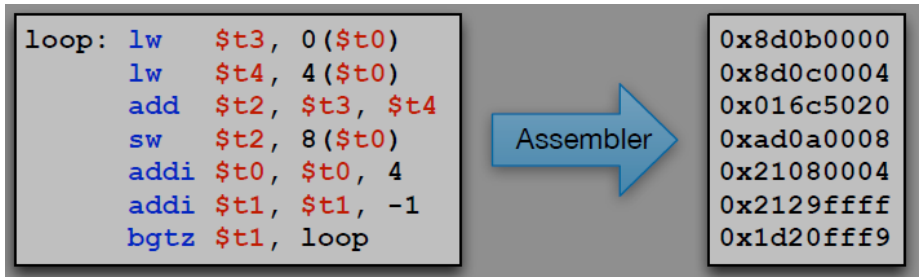
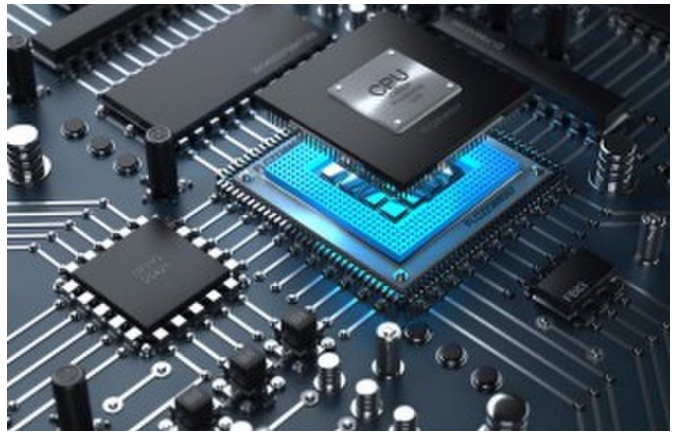




Lecture 10:
Intro to Functions in MIPS Assembly

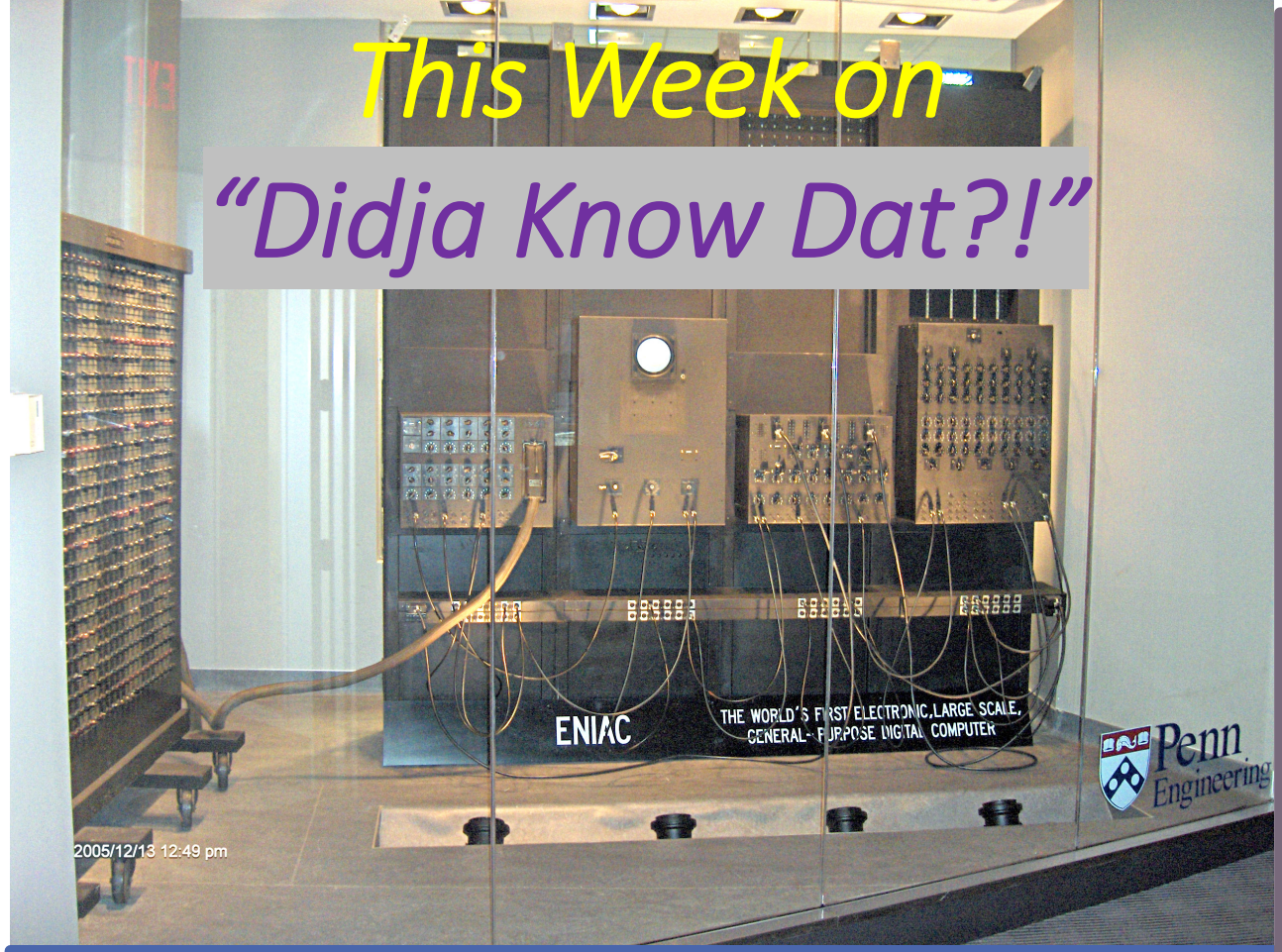
CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

This Week on

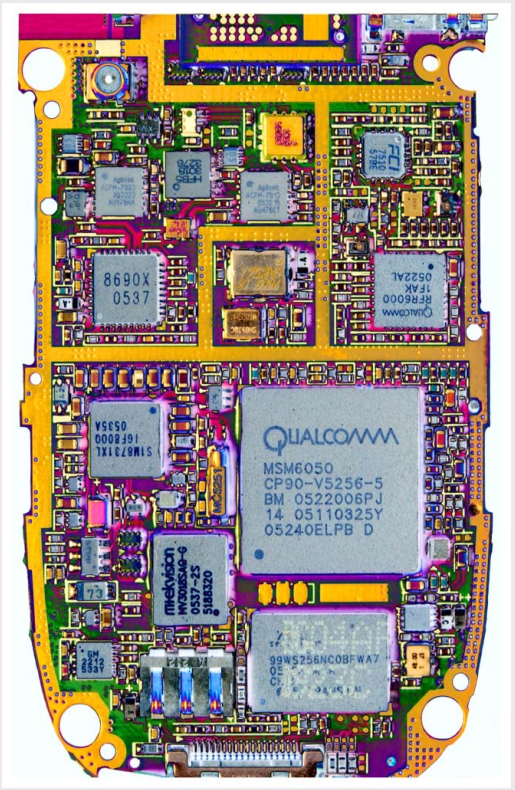
"Didja Know Dat?!"



One of the first *programmable* computers ever built for general and commercial purposes was the Electronic Numerical Integrator and Computer (ENIAC) in 1945.

It was 27 tons and took up 1800 square feet.
It used 160 kW of power (about 3000 light bulbs worth)
It cost $6.3 million in today's money to purchase.

Comparing today's cell phones (with dual+ CPUs), with ENIAC, we see they…



cost 17,000X less
are 40,000,000X smaller
use 400,000X less power
are 120,000X lighter
AND…
are 1,300X more powerful.

Administrative

Read **Handout 4?**

New assignment #5 (in **TWO** parts) is due on Monday!

A worksheet file is available for today's lecture on Canvas!

REMINDER: Come to our office hours!! ☺

REMINDER: Lab on Friday

Assembling Exercise

Let's find out what the MIPS assembly instruction is, given its machine language word: **0x3044FFEF**

STEPS TO TAKE!	
Turn this hex into a binary	
Identify the opcode first (where is that located??)	
Determine the format type & its fields (+ # of bits)	
Get field values Determine assembly instruction	

Assembling Exercise

Let's find out what the MIPS assembly instruction is, given its machine language word: **0x3044FFEF**

STEPS TO TAKE!	
Turn this hex into a binary	0011 0000 0100 0100 1111 1111 1110 1111
Identify the opcode first (where is that located??)	First 6 bits = 001100 = **0xC** ➔ the instruction is **andi** (look up in MIPS Sheet)
Determine the format type & its fields (+ # of bits)	**I-Type**: **op** (6b), **rs** (5b), **rt** (5b), **immed** (16b)
Get field values Determine assembly instruction	rs = 00010 = 2 ➔ rs = $v0 rt = 00100 = 4 ➔ rt = $a0 immed = 1111111111101111 = -(0000000000010001) = -17 **andi $a0, $v0, -17**

BTW: There is a fantastic project out of this (equiv) course at Bucknell College: https://www.eg.bucknell.edu/~csci320/mips_web/

iClicker!

Which of the following is the hex representation of the instruction:

`or $t9, $t9, $t8`

A. 0x03739825

B. 0x0338C825

C. 0x0339C025

D. 0x0319C825

An R-type:

Op.	**6b**	or => 000000
Rs	**5b**	$t9 => 25 = 11001
Rt	**5b**	$t8 => 24 = 11000
Rd	**5b**	$t9 => 25 = 11001
Shamt	**5b**	00000
Funct.	**6b**	0x25 = 100101

```
00000011001110001100100000100101

0x0  3   3   8   C   8   2    5
```

iClicker!!

Which of the following is equivalent to **160 kibi bits**?

A. 2^{14} bytes

B. 20×10^{10} bytes

C. 20×2^{10} bytes

D. 160×2^{10} bytes

E. 20^{10} bytes

160 kibi bits

= 20 kibi bytes

= 20×2^{10} bytes

Topics on Functions in MIPS

```
int mahFunkshun( int a ) {
    int b = 42 + a;
    return b;
}
…
…

int x = -42;
int y = mahFunkshun(x);
cout << y;
…
```

***How* do functions in MIPS work?**

- Jumping to a routine… and then jumping back…!
- Passing argument values into the function
- Getting returned values from the function
- What values should be *preserved (i.e., not changed)* when you call a function?
- How do we deal with *nested functions* and *recursive functions*?

Re-introduciiiiiiing…. ***The Stack***!

Worksheet

Implementing Functions

What capabilities in programming do we need to have to create/use functions?

1. Ability to ***call*** a function and ***execute code elsewhere*** in the program
   - We know how branches and jumps could help here

2. Way to ***pass arguments into*** function and get ***returns from*** function
   - Can do this in different ways – HOWEVER…
   - … there's a *preferred* way to do it (a.k.a a **convention**)

Jumping to Function Code

```
int doubleIt (int num) {
    return 2*num;
}

…

int n1 = 21;
int n2 = doubleIt(n1);
int n3 = (n2 + 3)*7;
…
```

We know how to jump *unconditionally* to another part of the program
(the `j` instruction)

But what about ***jumping*** to some function code, doing stuff there,
and then jumping back again to continue running your program?

- That is, **how do you know *where* to jump back to**?
- We'll need a way to *save* where we were before we jump to the function
  - Leave breadcrumbs!!!! (bad analogy) Put a bookmark!!! (better analogy)

Q: What do need so that we can do this on MIPS?

A: A way to store the value in the program counter (**$PC**) *before* we jump to the function --- ***why??***

Worksheet

Calling Functions on MIPS

Using **branching** and `j` is **not** good enough!!

Two crucial instructions: `jal` and `jr`

One specialized register: `$ra` (short for **r**eturn **a**ddress)

`jal` (**jump-and-link**)

- Simultaneously do 2 things: **jump to an address**, and **store the location of the next instruction** in register `$ra`

`jr` (**jump-register**)

- **Jump to an address stored in a register**, usually that's `$ra`

Worksheet

DEMO!

Simple Call Example

See program: **simple_call.asm**

Functions should always have their own label

*Note: SPIM **always starts** execution at the line labeled "**main**", even if other labels exist before it*

*Functions should always have **jr $ra** at the end!*

*Functions should always be called with **jal label***

```
# Calls a function (test) which immediately returns
.text
test:   # do stuff
        # maybe return something to the 'caller'
        jr $ra

main:   # do stuff…
        # then call the test function
        jal test

exit:   # exit
        li $v0, 10
        syscall
```

Worksheet

Passing and Returning Values

As programmers, we want to be able to call *arbitrary* functions
without knowing the *implementation details* of those functions

So, we need to know our *pre-/post-conditions* (remember those?!!!)

- Pre-conditions: what are the arguments into this function?
- Post-conditions: what does this function do and what value(s) does it return?

Q: How might we achieve this in MIPS?

- A: The standard way is to designate specific registers for **arguments** and **return values**

Follow-up Q: As a **law** of programming???

- A: Not as a "law", but as a ***convention*** *(what's the difference???)*

Worksheet

Calling Convention for Passing and Returning Values in MIPS

Registers **`$a0`** thru **`$a3`**

- **Argument registers**, for passing function **arguments** to the function

Registers **`$v0`** and **`$v1`**

- **Return registers**, for passing **return values** from the function

What if we want to pass > 4 args?

- There are ways around that… but we won't discuss them in CS64…!

Worksheet

The MIPS Calling Convention (part 1)

This is the **convention** we want to use whenever we use **functions** in MIPS Assembly

So far, we've said this convention dictates that:

1. All arguments to the function must be in **$a0 - $a3** registers
2. All returned values from the function must be in **$v0 - $v1** registers

For example:

```
int MahFunkshun(int m, int f) {
    int x = m - 2*f;
    return x;
}
```

```
MahFunkshun:  # m is $a0, f is $a1 (arguments)
    sll $a1, $a1, 1
    sub $v0, $a0, $a1    # x is $v0 (return value)
    jr $ra    # functions should end with jr instr.
```

But there's a LOT MORE to the convention than this!!!

Worksheet

Remember This from CS 16!!!

When we call a function in C++, what is our expectation of what happens to variable values?

```
int MahFunkshun(int m, int &f) {
    f = 2*f;
    int x = m - f;
    return x;
}

int main() {
    int a = 5, b = 2, c;
    int x = 99;
    c = MahFunkshun(a, b);
    cout << a << b << x << c;
}
```

WHAT VALUES DOES THIS PRINT???

***Answer**: 5, 4, 99, 1*

NOTE:

* variable **x** in main **!=** variable **x** in the function

TRUE or FALSE:

* variables **m**, **f**, **x** and their values, have no presence outside of the function **MahFunkshun**

Variable **f** is *passed by reference;* therefore, any changes made to it in **MahFunkshun**, will be observed in **main()**

Worksheet

Function Calls Within Functions…

```
void foo() {
  bar();
}
void bar() {
  baz();
}
void baz() {}
```

Given what we've said so far… consider the code shown here.....>>

What about this code makes our previously discussed setup *not work*?

ANS: You would need **multiple copies of $ra**

You'd have to copy the value of **$ra** somewhere before calling another function

- Where, though? Other registers?? That's likely NOT ok… (*why?*)

So obviously, you have to use something else… in memory… can u guess where?

The STACK!

To Dos

Current assignment is due on Monday

Read the next handout (**Handout 5**) for next week!

Remember: **Lab on Friday**

</LECTURE>